Recitation-3

Marlon Alen I Flores
BSCS - 3A

CS311
2026-08-14

Quick note:

“Apologies for the sudden change in code structure, trying to emulate the program structure of a windows forms program proved to be a confusing an error-prone experience. As such I proceeded to refactor the code to more closely resemble standard Avalonia Programs (the ui framework for linux that I’m currently using). Especially since I am also using Avalonia for the UI of our project, I’d also rather use this as a learning experience.”

https://avaloniaui.net/

Scenarios

Login each usertype and showing the file maintenance menu

With Administrator


File Maintenance
View Logs
About
Logout
Accounts
Equipments
Username: admin
Usertype: ADMINISTRATOR

With Technical

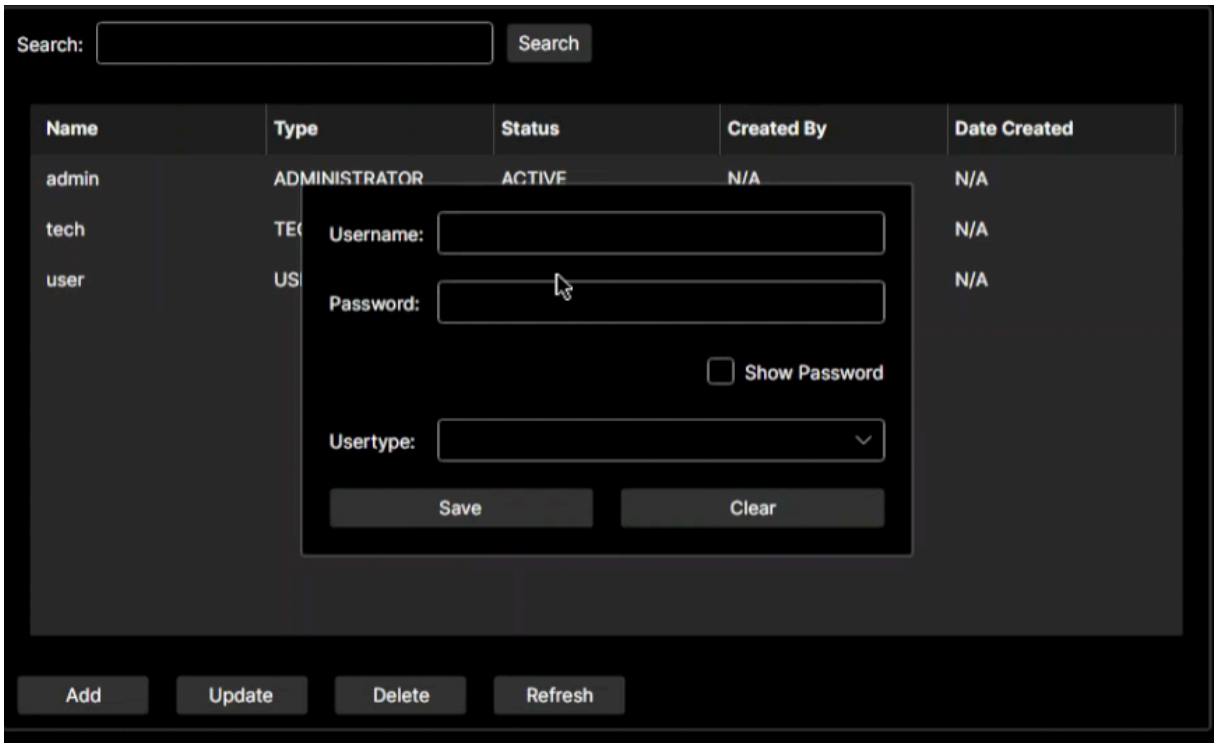

File Maintenance
View Logs
About
Logout
Equipments
Username: tech
Usertype: TECHNICAL

With User

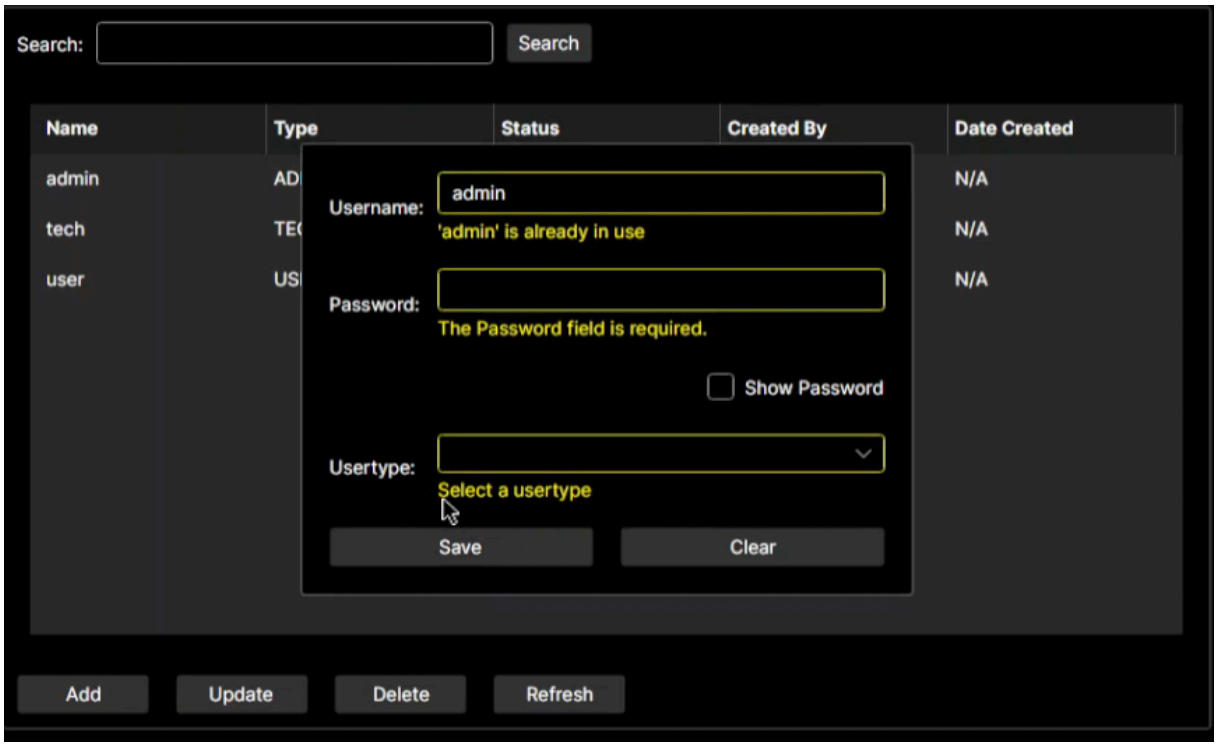

File Maintenance
View Logs
About
Logout
Username: user
Usertype: USER

Adding a new account and showing it on the accounts form.

Add Account Form

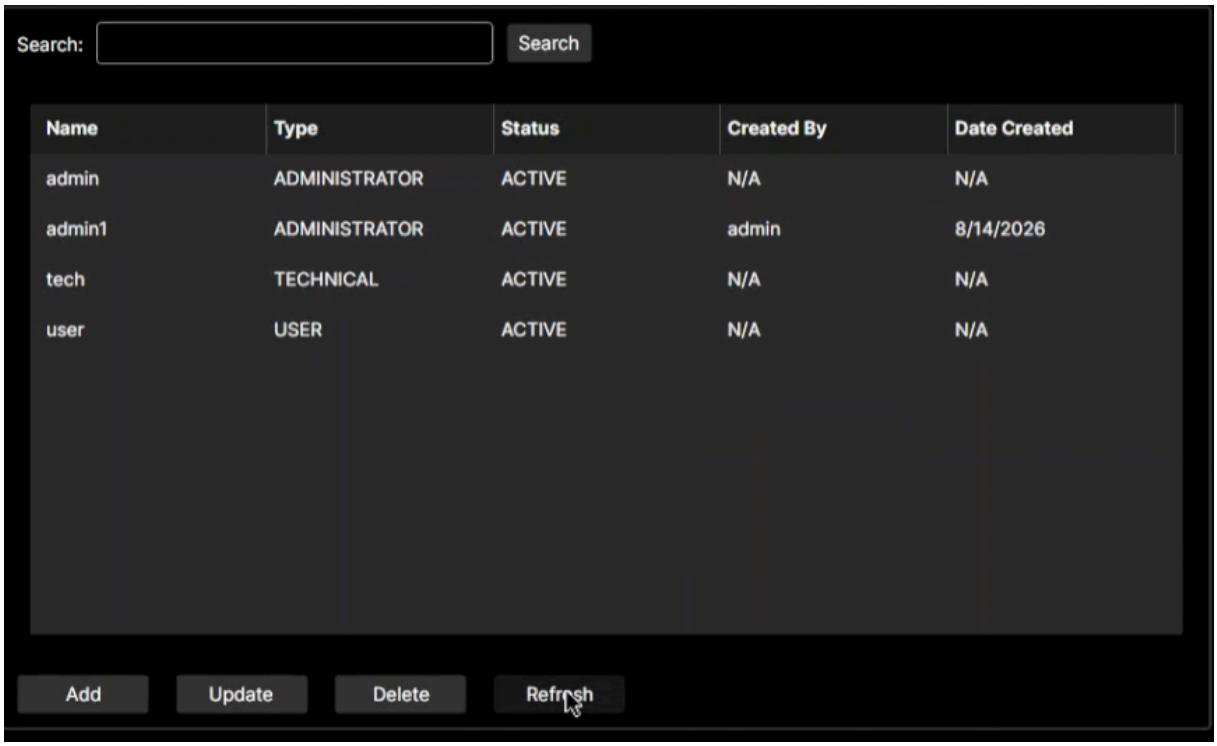


With Invalid Input



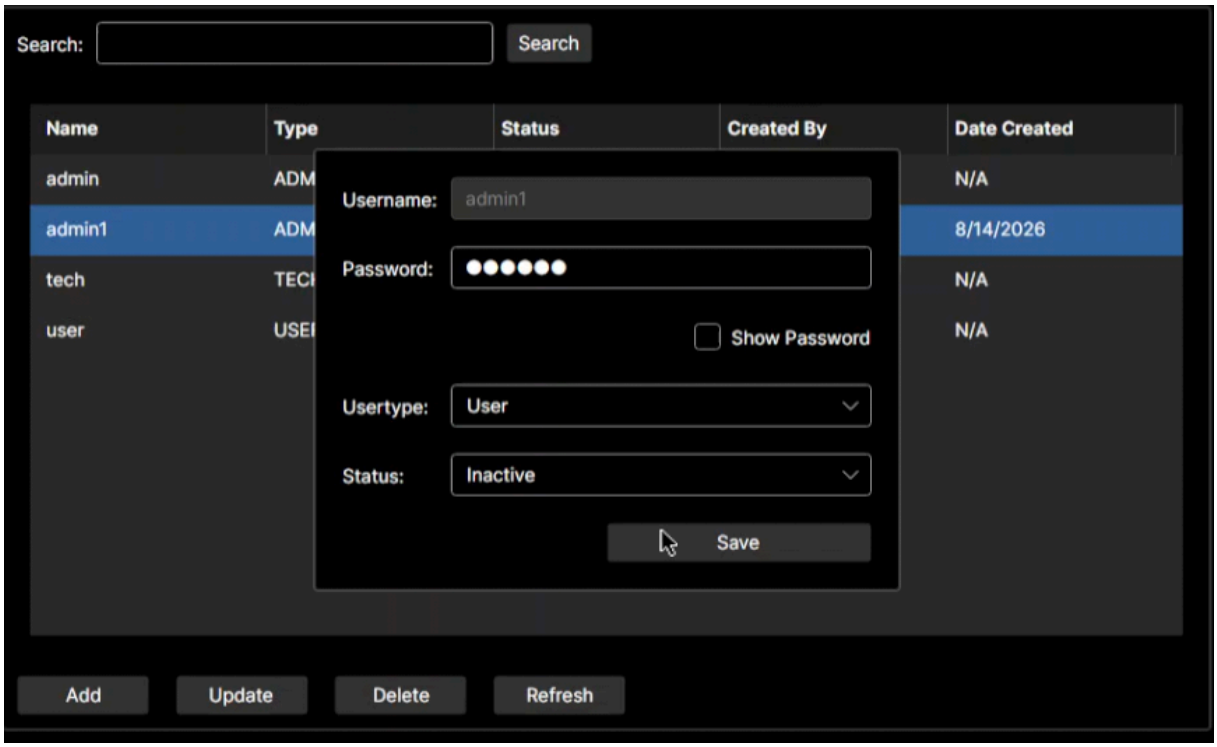
Account Added

Updated Table

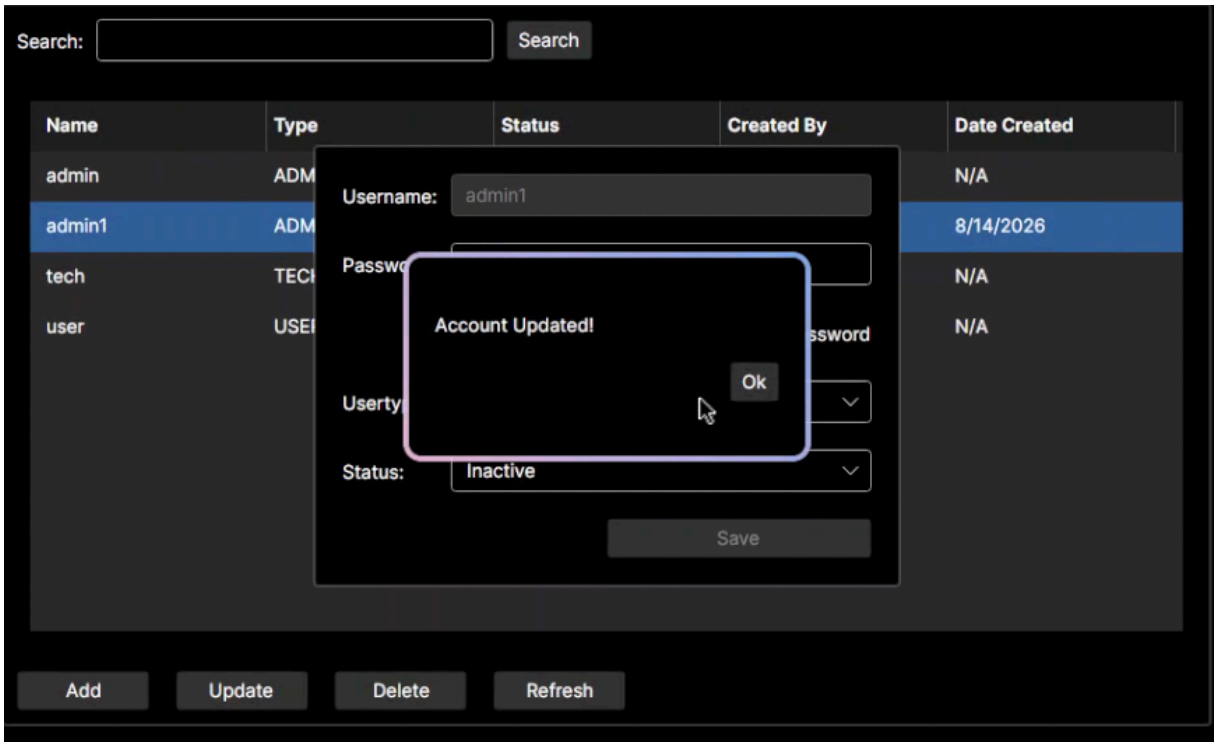


Updating the newly added account and showing it on the accounts form.

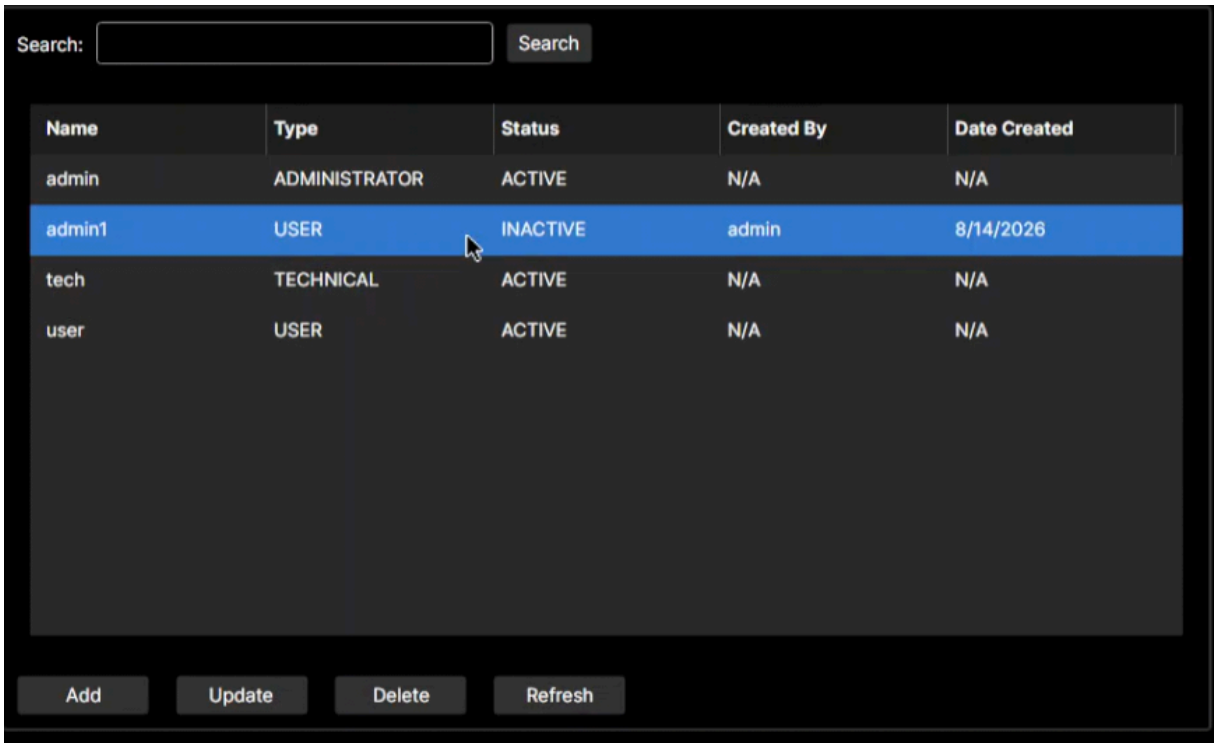
Update Account Form



Account Updated

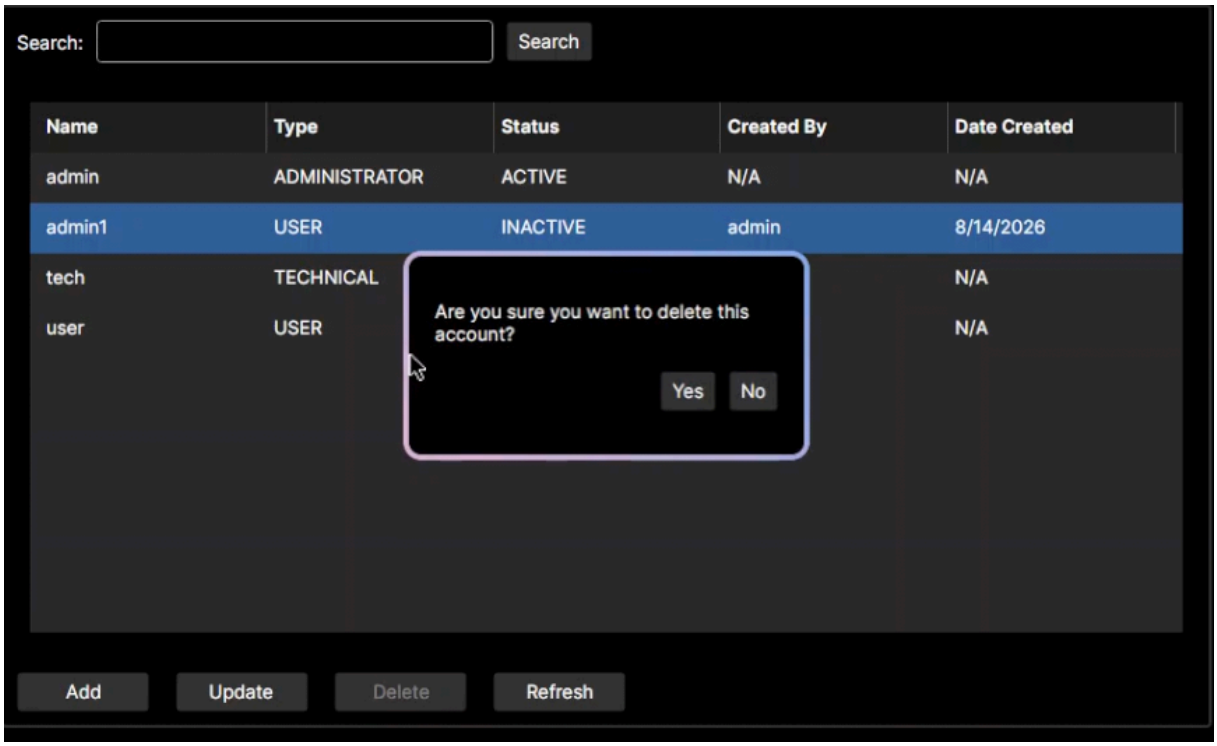


Updated Table

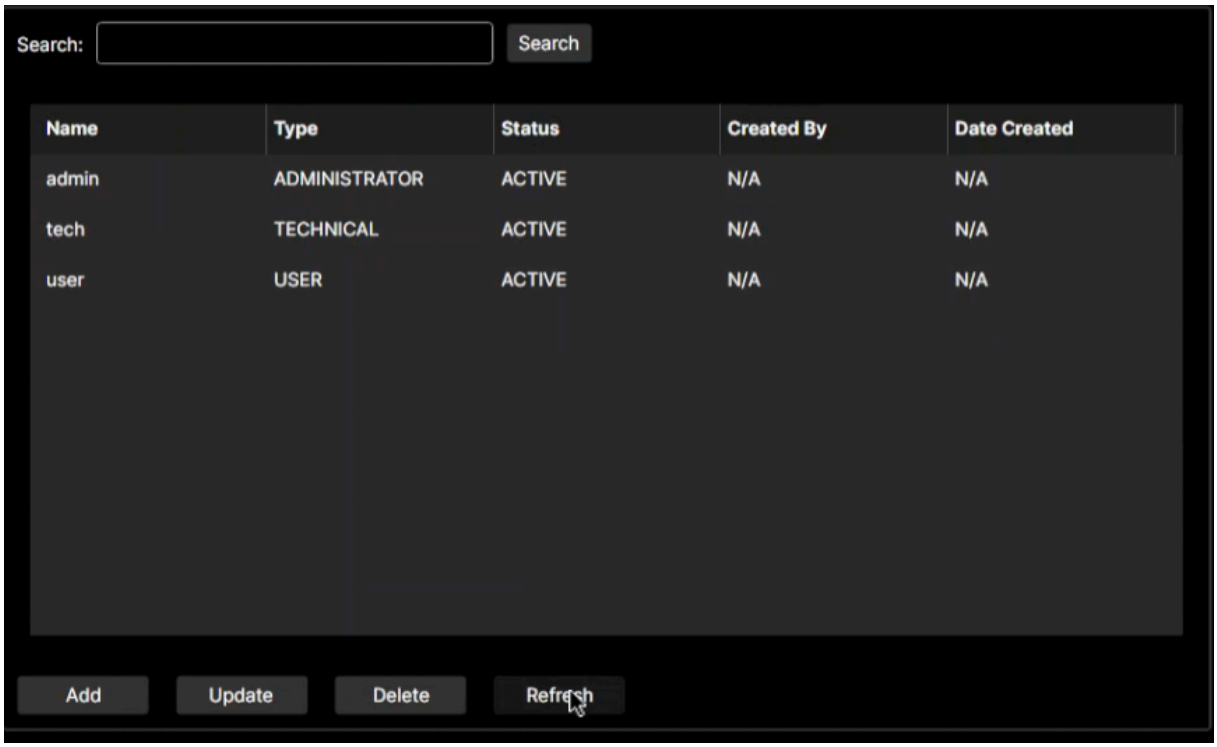


deleting the newly added account and showing the accounts form.

Delete Confirmation Prompt



Updated Table



Show the logs table.

datelog	timelog	action	module	performedby	performedto
14/08/2026	10:22 AM	Update account	Account Management	admin	admin1
14/08/2026	10:22 AM	Delete account	Account Management	admin	admin1

Screen Recording

https://drive.google.com/file/d/1KLSJ7PAbBXDtMvp2vpzWhRsqq6C1pgkJ/view?usp=sharing

Code

Full code is available at https://github.com/Alen-Flores/CS311-2026-FLORES/tree/master/midterm/CS311-Midterm-Recitation3-Flores

Models

./Models/Log.cs

```C#
public record Log(
   string datelog,
   string timelog,
   string action,
   string module,
   string performedby,
   string performedto
);
```

./Models/User.cs

```C#
public record User(
  string Username,
  string Password,
  string Usertype,
  string Status,
  string CreatedBy,
  string DateCreated
);
```

Services

./Services/DatabaseService.cs

```C#
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using ticket_management;

namespace CS311_CS3A_2026_Flores.Services;

public interface IDatabaseService
{
  User? GetUserByUsername(string username);
  User? GetUserByLogin(string username, string password);
  List<User> GetUsers();
  List<User> GetUsersByQuery(string query);
  bool AddUser(User user);
  bool UpdateUser(User user);
  void LogAction(Log log);
  bool DeleteUser(string username);
}

public class DatabaseService : IDatabaseService
{
  private Class1 DBContext;
  public DatabaseService()
  {
    DBContext = new Class1("127.0.0.1", "CS311-CS3A-2026-FLORES", "marlon",
    "flores");
  }

  public User? GetUserByUsername(string username)
  {
    DataTable dt = DBContext.GetData($"""
    SELECT * FROM tblaccounts
      WHERE username = '{username}'
```

```
        AND status = 'active'
    """);

    DataRow? row = dt.AsEnumerable().FirstOrDefault();
    if (row is null) return null;

    User user = new User(
      row.Field<string>("username")!,
      row.Field<string>("password")!,
      row.Field<string>("usertype")!,
      row.Field<string>("status")!,
      row.Field<string>("createdby")!,
      row.Field<string>("datecreated")!
    );

    return user;
  }

  public User? GetUserByLogin(string username, string password)
  {
    DataTable dt = DBContext.GetData($"""
    SELECT * FROM tblaccounts
      WHERE username = '{username}'
      AND status = 'active'
    """);

    DataRow? row = dt.AsEnumerable().FirstOrDefault();
    if (row is null) return null;

    if (password != row.Field<string>("password")!) return null;

    User user = new User(
      row.Field<string>("username")!,
      row.Field<string>("password")!,
      row.Field<string>("usertype")!,
      row.Field<string>("status")!,
      row.Field<string>("createdby")!,
      row.Field<string>("datecreated")!
    );

    return user;
  }

  public List<User> GetUsers()
  {
    DataTable dt = DBContext.GetData($"""
        SELECT username, password, usertype, status, createdby, datecreated
```

```
81              FROM tblaccounts
82              ORDER BY username
83          """);
84
85      return dt.AsEnumerable()
86          .Select(row => new User
87          (
88            row.Field<string>("username")!
89            , row.Field<string>("password")!
90            , row.Field<string>("usertype")!
91            , row.Field<string>("status")!
92            , row.Field<string>("createdby")!
93            , row.Field<string>("datecreated")!
94          )).ToList()
95          ;
96      }
97
98      public List<User> GetUsersByQuery(string query)
99      {
100       DataTable dt = DBContext.GetData($"""
101           SELECT username, password, usertype, status, createdby, datecreated
102             FROM tblaccounts
103             WHERE username like '%{query}%'
104                OR usertype like '%{query}%'
105             ORDER BY username
106         """);
107
108     return dt.AsEnumerable()
109         .Select(row => new User
110         (
111           row.Field<string>("username")!
112           , row.Field<string>("password")!
113           , row.Field<string>("usertype")!
114           , row.Field<string>("status")!
115           , row.Field<string>("createdby")!
116           , row.Field<string>("datecreated")!
117         )).ToList()
118         ;
119     }
120
121     public bool AddUser(User user)
122     {
123       DBContext.executeSQL($"""
124       INSERT INTO tblaccounts (username, password, usertype, status,
125                                createdby, datecreated)
126         VALUES (
127           "{user.Username}",
```

```
128           "{user.Password}",
129           "{user.Usertype}",
130           "{user.Status}",
131           "{user.CreatedBy}",
132           "{user.DateCreated}"
133         )
134       """);
135
136       return DBContext.rowAffected > 0;
137     }
138
139     public bool UpdateUser(User user)
140     {
141       DBContext.executeSQL($"""
142       UPDATE tblaccounts
143         SET password = "{user.Password}",
144             usertype = "{user.Usertype}",
145             status = "{user.Status}"
146         WHERE username = "{user.Username}"
147       """);
148       return DBContext.rowAffected > 0;
149     }
150
151     public void LogAction(Log log)
152     {
153       DBContext.executeSQL($"""
154       INSERT INTO tbllogs (datelog, timelog, action, module, performedby,
          performedto)
155         VALUES (
156           "{log.datelog}",
157           "{log.timelog}",
158           "{log.action}",
159           "{log.module}",
160           "{log.performedby}",
161           "{log.performedto}"
162         )
163       """);
164     }
165
166     public bool DeleteUser(string username)
167     {
168       DBContext.executeSQL($"""
169       DELETE from tblaccounts
170         WHERE username = "{username}"
171       """);
172
173       return DBContext.rowAffected > 0;
```

```
174     }
175 }
```

ViewModel

./ViewModels/LoginViewModel.cs

```C#
using System;
using System.ComponentModel.DataAnnotations;
using System.Threading.Tasks;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using CS311_CS3A_2026_Flores.Services;
using CS311_CS3A_2026_Flores.Views;

namespace CS311_CS3A_2026_Flores.ViewModels;

public partial class LoginViewModel : ObservableValidator
{

    public event Action<User>? OnLoginSuccess;

    private readonly IDatabaseService _dbService;

    public LoginViewModel(IDatabaseService dbService)
    {
        _dbService = dbService;
    }

    [ObservableProperty]
    [Required]
    private string username = "";

    [ObservableProperty]
    [Required]
    private string password = "";

    [RelayCommand]
    private async Task Login()
    {
        User? user = _dbService.GetUserByLogin(Username, Password);
        if (user is null || user.Status == "INACTIVE")
        {
            await Dialog.Show("Incorrect account details or account is inactive",
            Dialog.Buttons.Ok);
        }
        else
        {
```

```
41          Clear();
42          OnLoginSuccess?.Invoke(user);
43      }
44
45  }
46
47  [RelayCommand]
48  private void Clear()
49  {
50      Username = "";
51      Password = "";
52  }
53 }
```

./ViewModels/MainWindowViewModel.cs

```C#
1  using System;
2  using System.Threading.Tasks;
3  using CommunityToolkit.Mvvm.ComponentModel;
4  using CommunityToolkit.Mvvm.Input;
5  using CS311_CS3A_2026_Flores.Services;
6  using CS311_CS3A_2026_Flores.Views;
7
8  namespace CS311_CS3A_2026_Flores.ViewModels;
9
10 public partial class MainWindowViewModel : ObservableValidator
11 {
12
13
14     [ObservableProperty]
15     private User _User;
16
17     [ObservableProperty]
18     private ObservableObject? _currentPage;
19
20     private readonly IDatabaseService DbService;
21
22     public event Action? OnLogout;
23
24     public bool AccountsMenuItemVisible => User.Usertype == "ADMINISTRATOR";
25     public bool EquipmentsMenuItemVisible => true;
26
27     public MainWindowViewModel(User user, IDatabaseService dbService)
28     {
29         _User = user;
30         DbService = dbService;
31     }
```

```
32
33    [RelayCommand]
34    private async Task Logout()
35    {
36        var result = await Dialog.Show("Are you sure you want to log out?",
          Dialog.Buttons.YesNo);
37        if (result == Dialog.DialogResult.Yes)
38        {
39            OnLogout!.Invoke();
40        }
41    }
42
43    [RelayCommand]
44    private async Task Accounts() => CurrentPage = new AccountsViewModel(User,
      DbService);
45 }
```

./ViewModels/AccountsViewModel.cs

```C#
1  using System;
2  using System.Collections.ObjectModel;
3  using System.Linq;
4  using System.Threading.Tasks;
5  using CommunityToolkit.Mvvm.ComponentModel;
6  using CommunityToolkit.Mvvm.Input;
7  using CS311_CS3A_2026_Flores.Services;
8  using CS311_CS3A_2026_Flores.Views;
9
10 namespace CS311_CS3A_2026_Flores.ViewModels;
11
12 public partial class AccountsViewModel : ObservableObject
13 {
14     [ObservableProperty]
15     private ObservableCollection<User> _users;
16
17     [ObservableProperty]
18     private ObservableObject? _currentControl;
19
20     [ObservableProperty]
21     private int _selectedIndex = -1;
22
23     private readonly IDatabaseService dbService;
24     private readonly User user;
25
26     public AccountsViewModel(User user, IDatabaseService dbService)
27     {
28         this.dbService = dbService;
29         Users = new ObservableCollection<User>(dbService.GetUsers());
```

```
30      this.user = user;
31    }
32
33    [RelayCommand]
34    private async Task Search(string query)
35    {
36      if (query.Length == 0)
37      {
38        Users = new ObservableCollection<User>(dbService.GetUsers());
39      }
40      else
41      {
42        Users = new ObservableCollection<User>(dbService.GetUsersByQuery(query));
43      }
44    }
45
46    [RelayCommand]
47    private async Task Add()
48    {
49      var accountViewModel = new AddAccountViewModel(user, dbService);
50      CurrentControl = accountViewModel;
51      accountViewModel.OnClose += () =>
52      {
53        CurrentControl = null;
54      };
55    }
56
57    [RelayCommand]
58    private async Task Update()
59    {
60      User? selectedUser = Users.ElementAtOrDefault(SelectedIndex);
61      if (selectedUser is null) return;
62
63      var updateAccountViewModel =
64        new UpdateAccountViewModel(user.Username, selectedUser, dbService);
65      CurrentControl = updateAccountViewModel;
66      updateAccountViewModel.OnClose += () =>
67      {
68        CurrentControl = null;
69      };
70    }
71
72    [RelayCommand]
73    private void Refresh()
74    {
75      Users = new ObservableCollection<User>(dbService.GetUsers());
76    }
```

```
77
78    [RelayCommand]
79    private async Task Delete()
80    {
81      User? selectedUser = Users.ElementAtOrDefault(SelectedIndex);
82      if (selectedUser is null) return;
83
84      var dr = await Dialog.Show("Are you sure you want to delete this account?",
      Dialog.Buttons.YesNo);
85      if (dr == Dialog.DialogResult.Yes)
86      {
87        if (dbService.DeleteUser(selectedUser.Username))
88        {
89
90          dbService.LogAction(new Log(
91            DateTime.Now.ToString("dd/MM/yyyy"), DateTime.Now.ToString("h:mm
            tt") // ToShortTimeString causes an error
92            , "Delete account", "Account Management", user.Username,
            selectedUser.Username
93            ));
94          await Dialog.Show("Account Deleted!", Dialog.Buttons.Ok);
95        }
96      }
97
98    }
99  }
```

./ViewModels/AddAccountViewModel.cs

```
1   using System;                                                          C#
2   using System.Collections.ObjectModel;
3   using System.ComponentModel.DataAnnotations;
4   using System.Threading.Tasks;
5   using CommunityToolkit.Mvvm.ComponentModel;
6   using CommunityToolkit.Mvvm.Input;
7   using CS311_CS3A_2026_Flores.Services;
8   using CS311_CS3A_2026_Flores.Views;
9
10  namespace CS311_CS3A_2026_Flores.ViewModels;
11
12  public partial class AddAccountViewModel : ObservableValidator
13  {
14    private readonly User user;
15    private IDatabaseService dbService;
16
17    public Action? OnClose;
18
19    [ObservableProperty]
```

```
20    [CustomValidation(typeof(AddAccountViewModel),
      nameof(ValidateUniqueUsername))]
21    [Required]
22    private string _username = "";
23
24    [ObservableProperty]
25
26    [Required]
27    private string _password = "";
28
29    [ObservableProperty]
30    public ObservableCollection<string> _usertypes = new()
31    {
32      "Administrator",
33      "Technical",
34      "User",
35    };
36
37    [ObservableProperty]
38    [Required(ErrorMessage = "Select a usertype")]
39    private string? _selectedType;
40
41    public AddAccountViewModel(User user, IDatabaseService dbService)
42    {
43      this.user = user;
44      this.dbService = dbService;
45    }
46
47    public static ValidationResult? ValidateUniqueUsername(
48      string username,
49      ValidationContext context
50      )
51    {
52      var viewModel = (AddAccountViewModel)context.ObjectInstance;
53      if (string.IsNullOrWhiteSpace(username))
54        return ValidationResult.Success;
55
56      if (viewModel.dbService.GetUserByUsername(username) != null)
57        return new ValidationResult($"'{username}' is already in use");
58
59      return ValidationResult.Success;
60    }
61
62    [RelayCommand]
63    private async Task Clear()
64    {
65      Username = "";
```

```
66      Password = "";
67      SelectedType = null;
68      ClearErrors();
69    }
70
71    [RelayCommand]
72    private async Task Save()
73    {
74      ValidateAllProperties();
75      if (HasErrors) return;
76
77      var result = await Dialog.Show("Are you sure you want to make this
        account?",
78                  Dialog.Buttons.YesNo);
79      if (result == Dialog.DialogResult.Yes)
80      {
81        User newUser = new User(
82          Username, Password, SelectedType!.ToUpper(), "ACTIVE", user.Username,
83          DateTime.Now.ToShortDateString()
84        );
85        if (dbService.AddUser(newUser))
86        {
87          await Dialog.Show("New Account Added.", Dialog.Buttons.Ok);
88          Close();
89        }
90      }
91    }
92
93    [RelayCommand]
94    private void Close()
95    {
96      OnClose?.Invoke();
97    }
98  }
```

./ViewModels/UpdateAccountViewModel.cs

```C#
1   using System;
2   using System.Collections.ObjectModel;
3   using System.ComponentModel.DataAnnotations;
4   using System.Globalization;
5   using System.Threading.Tasks;
6   using CommunityToolkit.Mvvm.ComponentModel;
7   using CommunityToolkit.Mvvm.Input;
8   using CS311_CS3A_2026_Flores.Services;
9   using CS311_CS3A_2026_Flores.Views;
10
11  namespace CS311_CS3A_2026_Flores.ViewModels;
```

```
12
13  public partial class UpdateAccountViewModel : ObservableValidator
14  {
15
16    private IDatabaseService dbService;
17    private User target;
18    public Action? OnClose;
19
20    public string Username => target.Username;
21
22    [ObservableProperty]
23    [Required]
24    public string? _password;
25
26    [ObservableProperty]
27    public ObservableCollection<string> _usertypes = new()
28    {
29      "Administrator",
30      "Technical",
31      "User",
32    };
33
34    [ObservableProperty]
35    public ObservableCollection<string> _statuses = new()
36    {
37      "Active",
38      "Inactive"
39    };
40
41    [ObservableProperty]
42    [Required(ErrorMessage = "Select a usertype")]
43    private string? _selectedType;
44
45    [ObservableProperty]
46    [Required(ErrorMessage = "Select a status")]
47    private string? _selectedStatus;
48    private readonly string creator;
49
50    public UpdateAccountViewModel(string creator, User target, IDatabaseService
      dbService)
51    {
52      this.dbService = dbService;
53      this.target = target;
54      this.creator = creator;
55      Password = target.Password;
56      SelectedType =
      CultureInfo.CurrentCulture.TextInfo.ToTitleCase(target.Usertype.ToLower());
```

```
57      SelectedStatus =
        CultureInfo.CurrentCulture.TextInfo.ToTitleCase(target.Status.ToLower());
58    }
59
60    [RelayCommand]
61    private void Close()
62    {
63      OnClose?.Invoke();
64    }
65
66    [RelayCommand]
67    private async Task Save()
68    {
69      ValidateAllProperties();
70      if (HasErrors) return;
71
72      var result = await Dialog.Show("Are you sure you want to update this
        account?",
73                  Dialog.Buttons.YesNo);
74      if (result == Dialog.DialogResult.Yes)
75      {
76        User updatedUser = new User(
77          target.Username,
78          Password!, SelectedType!.ToUpper(), SelectedStatus!.ToUpper(),
79          target.CreatedBy, target.DateCreated
80        );
81        if (dbService.UpdateUser(updatedUser))
82        {
83          dbService.LogAction(new Log(
84            DateTime.Now.ToString("dd/MM/yyyy"), DateTime.Now.ToString("h:mm
              tt") // ToShortTimeString causes an error
85            , "Update account", "Account Management", creator, target.Username
86            ));
87
88          await Dialog.Show("Account Updated!", Dialog.Buttons.Ok);
89          Close();
90        }
91      }
92
93    }
94  }
```